Lab 12
Pointer II

Andy, Yu-Guang Chen
Assistant Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw







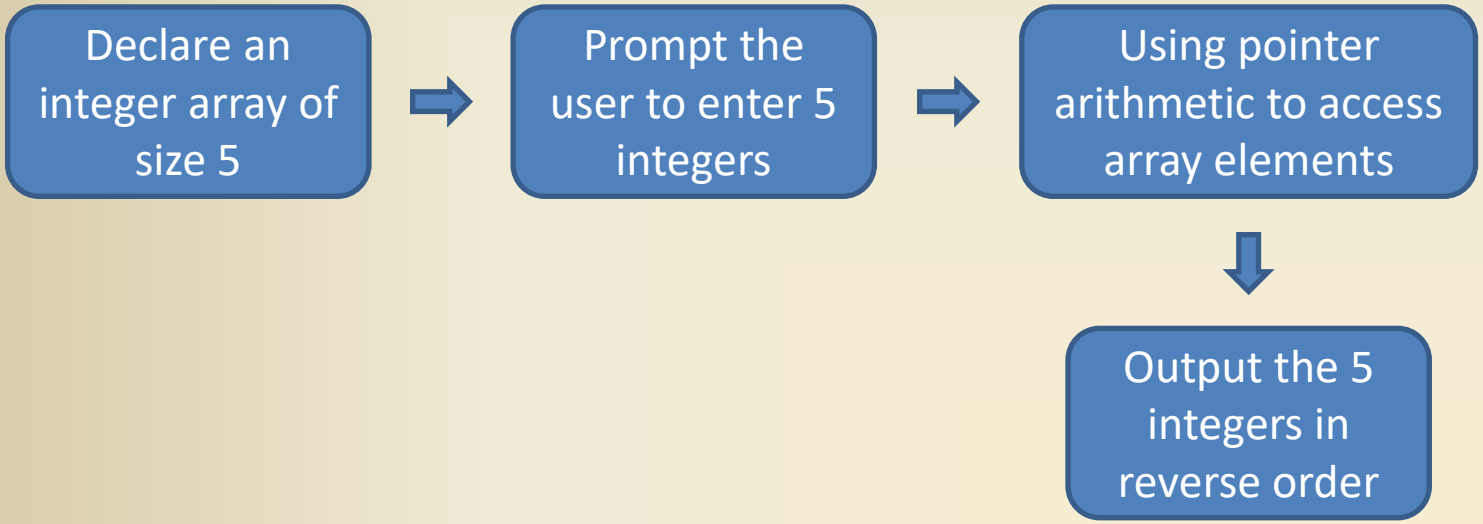
Lab 12.1
P113041 Reverse Output of an Array(30%)

- Test target: pointer
- Design a program that reads 5 integers from the user and outputs these numbers in reverse order.
- The data must be stored and transfered in an array.
- You must use pointer whenever you try to access any array in the program.



Lab 12.1
P113041 Reverse Output of an Array(30%)

- Flow chart:



Lab 12.1
P113041 Reverse Output of an Array(30%)

- *(b + i) is equivalent to b[i]

```
for (int i = 0; i < 5; i++) {
    cin >> *(b + i);
}
```

- Since i goes from 4 down to 0, we print b[4], b[3], b[2], b[1], and b[0] in that order

```
for (int i = 4; i >= 0; i--) {
    cout << *(b + i) << " ";
}
```







Lab 12.2
P113042 Digital Root(30%)

- Test target: pointer
- You are asked to write a function *void digitalRoot(int num, int *dRoot)* to calculate and return the digital root of the number.
- You can not change anything but the block between // CODE BEGIN and // CODE END in the code template.
- In main function, you can ONLY call *void digitalRoot(int num, int *dRoot),* no other functions are allowed. And you can ONLY output the final result in main function.



Lab 12.2
P113042 Digital Root(30%)

◆ Flow Chart:

The main program reads an integer num from input.

It sets *dRoot* to 0 and calls *digitalRoot(num, dRoot),* which sums the digits of num.

After the call, num is set to *dRoot*. If *dRoot* is 10 or more, it repeats the process (reset *dRoot* to 0 and call *digitalRoot* again).

This continues until *dRoot* is a single-digit number (< 10), at which point it prints out the final digital root.



Lab 12.2
P113042 Digital Root(30%)

- main()

```
    do {
        dRoot = 0;
        digitalRoot(num, dRoot);
        num = dRoot;
    } while (dRoot >= 10);

    cout << "digital root: " << dRoot << endl;

    return 0;
}
```

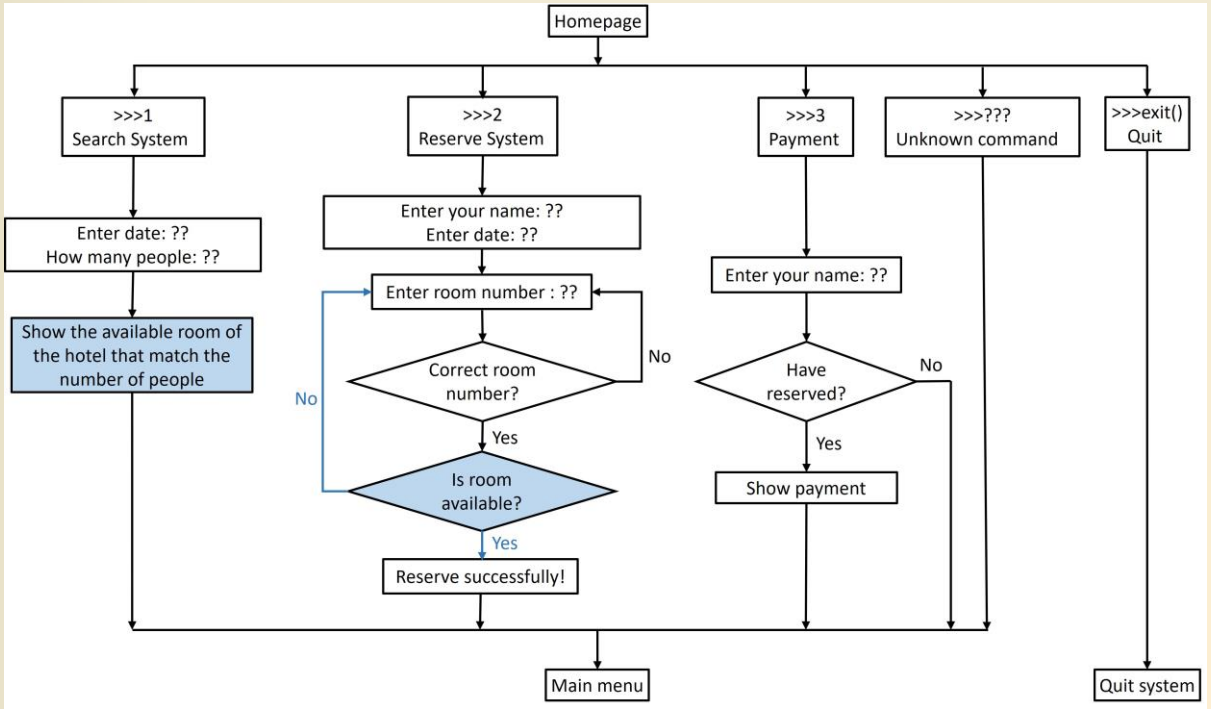
- *void digitalRoot()*

```
void digitalRoot(int num, int &dRoot) {
    while (num > 0) {
        dRoot += num % 10;
        num /= 10;
    }
}
```



Lab 12.3
P113043 Booking System II(30%)

- Test target: function
- You are asked to write a program to expend the system, Lab11.3. Now, the number of each type of the room is limited to just 1.



Lab 12.3
P113043 Booking System II(30%)

- Rooms become unavailable once reserved
  - In Lab 11.3, there was no mechanism to check if a room had already been reserved. Users could repeatedly select any room, regardless of its availability. In Lab 12.3, a new data structure *not_available (vector<vector<string>>)* is introduced to keep track of which rooms have been reserved for specific dates.
  - In the **Reserve System**, before accepting a reservation, the program *check*s if the selected room on the given date already exists in *not_available.*
  - If the room is already reserved, the program informs the user and prompts them to select another room.







Lab 12.3
P113043 Booking System II(30%)

- System only displays available rooms
  - In Lab 11.3, the Search System simply displayed all rooms matching the number of people entered, regardless of whether those rooms were already reserved. In Lab 12.3, the program checks the not_available data structure before displaying room options.
  - If a room has been reserved for the selected date, it will not be shown in the Search System.
  - If all the rooms in a specific hotel are fully booked, the hotel name and its room list will not appear at all.
  - This ensures that users only see available rooms when searching for accommodations.



Lab 12.3
P113043 Booking System II(30%)

- Reserve System
  - Introducing data structure *not_available*

```
while (true) {
    cout << "Enter room number to reserve: ";
    cin >> input;

    if (input == "1-1" || input == "1-2" || input == "2-1" || input == "2-2" || input == "2-3" ||
        input == "3-1" || input == "3-2" || input == "3-3") {

        for (int i = 0; i < not_available.size(); i++) {
            if (date == not_available[i][0] && input == not_available[i][1]) {
                remain = 0;
                cout << "The selected room is not available, please enter again!" << endl;
                break;
            }
        }

        if (remain) {
            temp2.push_back(date);
            temp2.push_back(input);
            not_available.push_back(temp2);

            temp2.clear();

            break;
        }


        remain = 1;
    }

    else
        cout << "Wrong number, please enter again!" << endl;
}

cout << "Reserved successfully!" << endl;
```

Lab 12.4
P113044 Maze(20%)

- Test target: pointer
- You are asked to write a program to solve the maze. The maze will be formed by '0' and '1'. '0' means the way you can walk, '1' means the wall. The entrance of the maze is '0', and it must be at the upper left corner (ex. (0,0) as below); the exit is '0', and it must be at the lower right corner (ex. (3,5) as below).



Lab 12.4
P113044 Maze(20%)

- Pathfinding Algorithm
  - Initialization:
    - Set all values in mark to 0 (unvisited).
    - Mark the starting point (1,1) as visited: mark[1*(colCount+2) + 1] = 1.
    - Initialize the stack with the starting position and direction.
  - Main Loop: The while loop continues until either the stack is empty (top == -1) or the exit is found (found = 1).
    - Pop: Retrieve the current position (row, col) and direction dir from the top of the stack using the pop function.



Lab 12.4
P113044 Maze(20%)

- Explore Directions: In the while (dir < 4 && !found) loop:
  - Calculate the next position (next_row, next_col) using move[dir].
  - Compute the corresponding maze and mark indices:
    nextMazeIndex = next_row(colCount+2) + next_col.*
  - Check Exit: If *(next_row, next_col)* is the exit, push the current and exit positions onto the stack, set found = 1, and break.
  - Check Valid Move: If the next position is walkable *(maze[nextMazeIndex] == 0)* and unvisited *(mark[nextMazeIndex] == 0)*:
    - » Mark the next position as visited.
    - » Push the current position and the incremented direction onto the stack.
    - » Move to the next position and reset dir to 0 (start checking from “up” again).
  - Else: Increment dir to check the next direction.
- If all directions are checked and none work, the function backtracks to the previous position using the stack.





Lab 12.4
P113044 Maze(20%)

- Output:
  - If the path is found *(found = 1),* print all the positions in the stack as the path.
  - If no path is found, print "Can't reach the exit!".



Lab 12.4
P113044 Maze(20%)

- Stack

```
void pop(int &top, int *row, int *col, int *dir, int *rowStack, int *colStack, int *dirStack) {
    *row = *(rowStack + top);
    *col = *(colStack + top);
    *dir = *(dirStack + top);
    top--;
}

void push(int &top, int row, int col, int dir, int *rowStack, int *colStack, int *dirStack) {
    top++;
    *(rowStack + top) = row;
    *(colStack + top) = col;
    *(dirStack + top) = dir;
}
```

Lab 12.4
P113044 Maze(20%)

- Pathfinding

```
void path(int *maze, int EXIT_ROW, int EXIT_COL, int colCount) {
    int row, col, next_row, next_col, dir, found = 0;

    // Stacks for row, column, and direction
    static int rowStackMem[MAX_SIZE * MAX_SIZE];
    static int colStackMem[MAX_SIZE * MAX_SIZE];
    static int dirStackMem[MAX_SIZE * MAX_SIZE];

    int *rowStack = rowStackMem;
    int *colStack = colStackMem;
    int *dirStack = dirStackMem;

    // Move directions stored as pointer-based access
    int moveMem[4][2] = {{-1, 0}, {0, 1}, {1, 0}, {0, -1}};
    int *move = &moveMem[0][0]; // Base pointer to the first element

    // Visited cells marker
    static int markMem[MAX_SIZE * MAX_SIZE];
    int *mark = markMem;

    // Initialize all cells in mark to 0 (unvisited)
    for (int i = 0; i < MAX_SIZE * MAX_SIZE; i++) {
        *(mark + i) = 0;
    }

    // Starting position (1,1), marked as visited
    *(mark + (1 * (colCount + 2) + 1)) = 1;

    int top = 0;
    *(rowStack + top) = 1;
    *(colStack + top) = 1;
    *(dirStack + top) = 1;
```

Lab 12.4
P113044 Maze(20%)

- Pathfinding

```
    while (top > -1 && !found) {
        pop(top, &row, &col, &dir, rowStack, colStack, dirStack);

        while (dir < 4 && !found) {
            next_row = row + *(move + dir * 2);       // row offset
            next_col = col + *(move + dir * 2 + 1);   // col offset

            int nextMazeIndex = next_row * (colCount + 2) + next_col;
            int nextMarkIndex = nextMazeIndex;

            if (next_row == EXIT_ROW && next_col == EXIT_COL) {
                found = 1;
                push(top, row, col, dir + 1, rowStack, colStack, dirStack);
                push(top, next_row, next_col, 0, rowStack, colStack, dirStack);
            } else if (*(maze + nextMazeIndex) == 0 && *(mark + nextMarkIndex) == 0) {
                *(mark + nextMarkIndex) = 1;
                push(top, row, col, dir + 1, rowStack, colStack, dirStack);
                row = next_row;
                col = next_col;
                dir = 0;
            } else {
                dir++;
            }
        }
    }

    if (found) {
        for (int i = 0; i <= top; i++) {
            int r = *(rowStack + i);
            int c = *(colStack + i);
            cout << "(" << r - 1 << "," << c - 1 << ") ";
        }
    } else {
        cout << "Can't reach the exit!";
    }
}
```





Evaluation

- Lab12.1(30%)
  - 5 testcases, 6% each
- Lab12.2(30%)
  - 5 testcases, 6% each
- Lab12.3(30%)
  - 5 testcases, 6% each
- Lab12.4(20%)
  - 5 testcases, 4% each







Andy, Yu-Guang Chen

Assistant Professor, Department of EE, NCU

Email: andyygchen@ee.ncu.edu.tw